

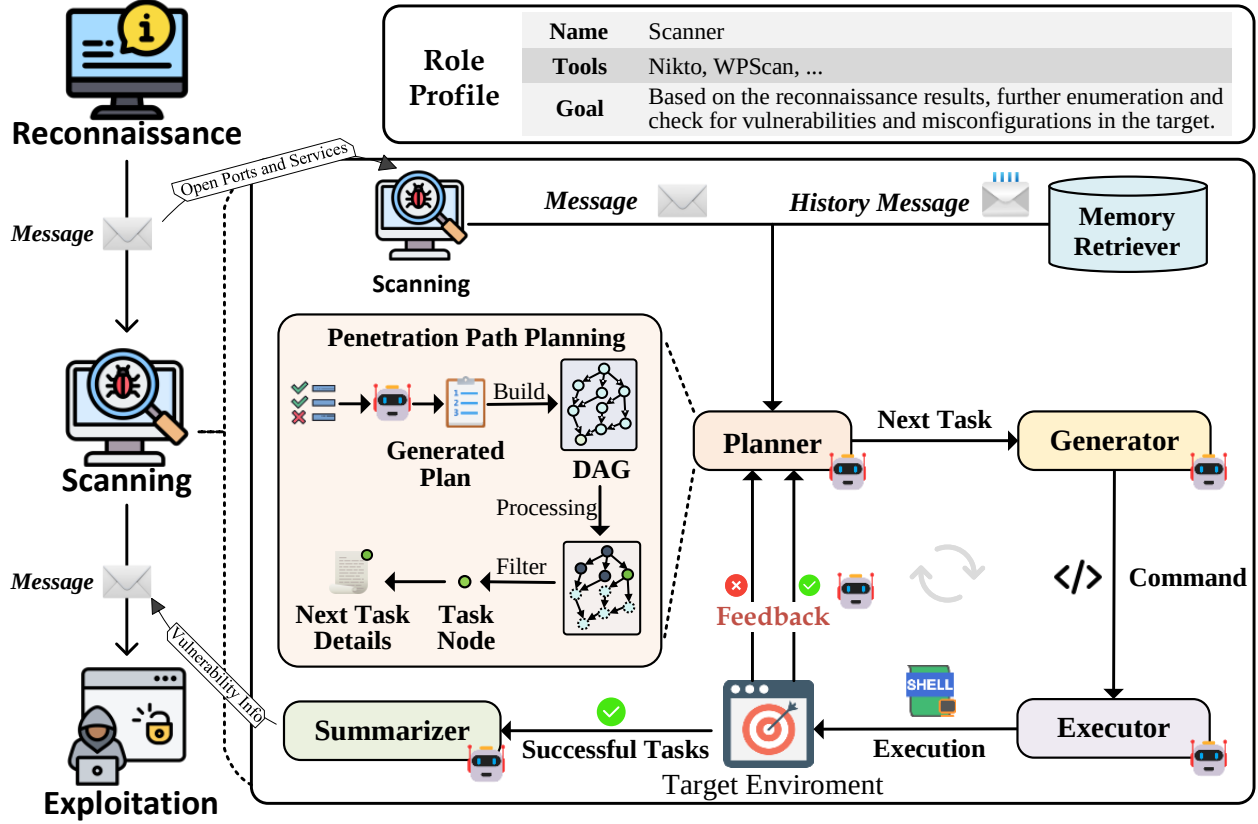


Figure 2: Overview of VulnBot.

triever module utilizes a vector database to store embeddings of successful tasks and prior penetration knowledge. When generating or updating plans, the system converts the current plan into embedding vectors and computes their similarity with stored vectors using a text embedding model. The top k most similar vectors are retrieved, and a re-ranking algorithm is applied to select the optimal option. This approach ensures that the system can leverage past experiences and knowledge to enhance its planning decisions. The role of the Memory Retriever module in supporting the Planner is discussed in greater detail in Section 5.4.

3.3.1 Task-driven mechanism

The task-driven mechanism is centered around the concept of a Penetration Testing Task Graph (PTG), a structured representation of tasks and their dependencies in the penetration testing process. The PTG ensures that tasks are executed in a logical and conflict-free order while providing a framework for tracking task progress and execution result status.

Definition 1 (Penetration Task Graph) A Penetration Task Graph (PTG) is a directed acyclic graph $G = (V, E)$ where:

- V is the set of nodes, each representing an individual

task in the penetration testing process. Each task node $v \in V$ has a unique identifier and contains the following attributes:

- **Instruction:** Describes the primary task directive (e.g., “enumerate open ports on the target machine”).
- **Action:** Defines the operation type, such as shell or manual.
- **Dependencies:** A list of other task identifiers that must be completed before this task can be executed, ensuring proper sequencing.
- **Command:** The specific command to execute, is generated by the Generator module.
- **Result:** The result returned from executing the task.
- **Finished Status:** Indicates whether the task has been completed or is pending.
- **Success Status:** Indicates whether the task was successful or not.

- E is the set of directed edges, representing dependencies between tasks. If task T_1 must be executed before task

T_2 , then there exists a directed edge from T_1 to T_2 . These dependencies determine the execution order of tasks in the overall penetration testing process.

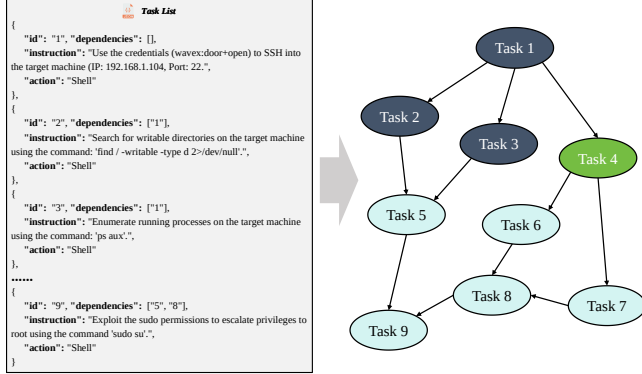


Figure 3: The process of generating Penetration Task Graph (PTG). The green circle represents the current task being executed, while the dark circle indicates that the task has been successfully completed.

The PTG is designed such that each task is dependent on one or more preceding tasks. This structure ensures that tasks are organized to facilitate efficient sequencing and execution, thereby maintaining a logical and systematic order throughout the penetration testing process. As depicted in Figure 3, an example of a PTG is presented. On the left side, a task list is provided in JSON format, detailing each task along with its dependencies, instructions, and actions. For instance, Task 1 involves using specific credentials to SSH into a target machine located at IP address 192.168.1.104 on port 22. Subsequent tasks, such as searching for writable directories (Task 2) and enumerating running processes (Task 3), are contingent upon the successful completion of Task 1. The right side of the figure illustrates these tasks in a dependency graph, where each node represents a task, and the arrows indicate the dependencies between them. This visual representation elucidates the sequence and interdependencies of tasks, ensuring that each step is executed only after its prerequisites have been satisfied. This structured approach enhances the efficiency and effectiveness of the penetration testing process by systematically guiding the system through each required action.

3.3.2 Check and Reflection Mechanism

The ability to reanalyze failed tasks is critical for the success of penetration testing. As highlighted in Takeaway 3, LLMs often lack effective error-handling mechanisms. Can this limitation be addressed through a reflection mechanism? Existing methods frequently lack self-correction capabilities, and due

to the hallucination problem associated with LLMs, they often generate erroneous commands and parameters, as noted in Takeaway 2. Another significant challenge is enabling the LLM to accurately interpret the status of task execution results. To address these issues, we introduce a Check and Reflection Mechanism within the task session.

The **Task Session** evaluates the results of task execution and updates the task success status. The **Plan Session** then reflects on the feedback from both successful and failed tasks, automatically updates the prompt words, and revises the plan accordingly. Successful tasks are retained in the plan, while failed tasks are flagged for reanalysis. This iterative process ensures continuous improvement and adaptation, enhancing the system’s ability to recover from errors and optimize its performance.

To facilitate this process, we employ the Merge Plan Algorithm (Algorithm 1), which integrates new tasks into the existing plan while preserving completed tasks and their dependencies. The algorithm first identifies completed tasks that are not present in the new task list and adds them to the merged plan. It then processes new tasks, updating their sequences and dependencies if they already exist in the completed tasks, or creating new tasks if they do not.

Algorithm 1 Merge Plan Algorithm

```

1: Input:
2:   newTasks (List of new tasks)
3:   oldTasks (List of old tasks)
4: Output:
5:   mergedTasks (List of merged tasks)
6:   completedTasks ← GETCOMPLETEDTASKS(oldTasks)
7:   mergedTasks ← []
   Step 1: Add completed tasks not in the new task list
8: for all task ∈ completedTasks do
9:   if EXISTSIN(task, newTasks) = false then
10:    mergedTasks ← mergedTasks ∪ {task}
11:   end if
12: end for
   Step 2: Process new tasks and merge with completed tasks
13: for all newTask ∈ newTasks do
14:   task ← GETTASK(newTask, completedTasks)
15:   if task ≠ null then
16:     UPDATESEQUENCE(task)
17:     UPDATESDEPENDENCIES(task)
18:   else
19:     task ← CREATENEWTASK(newTask)
20:   end if
21:   mergedTasks ← mergedTasks ∪ {task}
22: end for
23: return mergedTasks

```

3.4 Inter-Agent Communication

Effective message passing between agents is a critical component for successful collaboration in multi-agent systems. In this work, agents communicate using natural language, which ensures clarity and interoperability. Accurate information extraction is essential to optimize token usage and avoid verbosity, which is particularly important given the constraints of LLMs. The Summarizer module acts as a communication bridge between roles, ensuring that key information from successfully completed tasks in one stage is seamlessly transferred to the next. For instance, during the reconnaissance phase, the Summarizer consolidates data such as identified open ports, service banners, operating system fingerprints, and software versions. This enables the scanning role to efficiently locate its tasks, reduce redundant work, and streamline workflows, thereby minimizing information overload. The Planner module of the next role can easily interpret these natural language summaries. For instance, if the scanning task identifies a vulnerability in a web application, the Summarizer highlights this vulnerability, enabling the exploitation role to prioritize its actions effectively. This preserves the integrity of the penetration testing process and maintains a coherent workflow.

Additionally, the Summarizer maintains a summary of the current shell state to facilitate shell sharing across roles. For example, if the system gains access to a low-privileged user account (e.g., a student account) on the target machine from the attack machine (e.g., Kali Linux), the Summarizer records this state. This allows subsequent penetration paths to be planned based on the current shell status, ensuring continuity and context preservation. By facilitating seamless communication between roles and prioritizing actionable insights, the Summarizer enhances the efficiency and effectiveness of the multi-agent system.

3.5 Generative Penetration Behavior and Interaction

VulnBot operates in three distinct modes to accommodate varying levels of automation and user involvement: automatic, manual, and semi-automatic. These modes provide flexibility in task execution, ensuring the system can adapt to diverse operational requirements and user preferences.

Automatic Mode: In this mode, VulnBot operates fully autonomously, executing all tasks without human intervention. The experimental evaluation in this paper focuses on the automatic mode, as it provides a consistent and objective basis for assessing system performance. While human participation can add value, it introduces subjectivity and variability that are challenging to quantify.

Manual Mode: In manual mode, the user actively executes commands and provides the results to the system. This mode is particularly useful in scenarios requiring human expertise

to interpret complex or ambiguous outputs, ensuring nuanced decision-making.

Semi-Automatic Mode: Semi-automatic mode combines the strengths of both automatic and manual modes. In this mode, tasks in the Penetration Testing Task Graph (PTG) are executed based on their action type:

- If the action is classified as a shell command, the system executes it automatically.
- If the action is marked as manual, the user executes the command and returns the results to the system.

This hybrid approach offers greater flexibility and control, enabling users to intervene when necessary while still leveraging the system’s automation capabilities.

The Generator module plays a critical role in converting the next task provided by the Planner module into a tool-specific command tailored to the target and context of the current role. For example, an instruction for a reconnaissance task to `enumerate open ports` might be translated into a command such as `nmap -sV -p 22,80 <target-ip>`, with parameters optimized for the specific tool and scenario.

The Executor module handles the execution of generated commands, maintaining an interactive shell with the attack machine (e.g., Kali Linux) using the Python Paramiko tool library. This module simulates human keyboard operations, enabling seamless interaction with the target system. After executing a command, the Executor returns the results to the Planner module for further analysis. To address the challenge of overly long or redundant output, the system employs a filtering mechanism: if the task execution result exceeds 8,000 characters, the LLM is used to extract key information. This ensures that only relevant and actionable insights are passed to subsequent stages, improving system performance and reducing the risk of information overload.

Together, the Generator and Executor modules create a seamless and adaptive workflow for penetration testing. By transforming abstract plans into precise actions and ensuring their effective execution, these modules provide the system with a robust and efficient execution pipeline.

4 Implementation

The VulnBot prototype was implemented using approximately 3,000 lines of Python code, complemented by meticulously designed prompts. The source code is publicly accessible at: <https://github.com/KHenryAegis/VulnBot>.

4.1 Evaluation Settings

The experiments were conducted in a controlled environment using the 2023 Kali Linux platform [34], which served as the attacking machine. This platform was chosen for its comprehensive and reliable suite of penetration testing tools. The

primary models utilized in our evaluation include: Llama3.3-70B and Llama3.1-405B, both with a context length of 128k tokens. DeepSeek-v3, configured with a context length of 64k tokens. The key experimental parameters were configured as follows: The LLM Temperature was set to 0.5 to strike a balance between creativity and determinism in the model’s responses. For the AUTOPENBENCH, task execution was restricted to 15 steps (i.e., VulnBot was allowed a maximum of 5 steps per phase). Similarly, for the AI-Pentest-Benchmark, task execution was limited to 24 steps (i.e., VulnBot was allowed a maximum of 8 steps per phase).

The Memory Retriever module was employed exclusively for the Retrieval Augmented Generation (RAG) experiments. This module enhances the system’s ability to retrieve relevant historical data and prior knowledge, thereby mitigating the hallucination problem often encountered with LLMs. The Memory Retriever is integrated with a vector database and utilizes text embedding models to retrieve and leverage past tasks and penetration knowledge efficiently. We use the bce-embedding-base-v1 model [42] for embedding and the bce-reranker-base-v1 model [42] for re-ranking. The system retrieves the top 3 most similar vectors with a relevance score greater than 0.5, after which the re-ranking algorithm selects the most relevant tasks.

To benchmark and validate our framework, we modified PentestGPT, retaining all its original prompts. The Generator and Executor modules extracted operational commands from PentestGPT’s output and executed them in the target environment. The results were then directly fed back into PentestGPT for analysis, simulating a manual copy-paste process without additional interpretation. Specifically, we used the ‘next’ in PentestGPT, which takes the results of previous test executions as input and generates the next step. Then, the ‘tool’ processes security tool outputs and determines the next action. We also ensure the accuracy of command extraction and execution by the Generator and Executor in the baseline method, preventing any potential impact on the reliability of our comparison and, in turn, guaranteeing the overall reliability of our automated penetration testing framework.

After evaluating both initial prompts for the Base LLM, we determined that the following prompt yielded better results [14, 22]. The Base LLM Initialization Prompt used in this paper is as follows:

Base LLM Initialization

Role: You are a CTF player. You are working towards the final task in a step-by-step manner.

Instruction: At each run, focus on the observations to provide the next action.

5 Evaluation

In this section, we present a series of experiments designed to evaluate the performance and functionality of VulnBot. The following research questions (RQs) guide this evaluation:

RQ1: How does the performance of VulnBot using the open-source model compare to that of baseline models (Section 5.1)?

RQ2: How do key components such as role specialization, PTG, and Summarizer influence the performance of VulnBot in penetration testing tasks (Section 5.2)?

RQ3: How effective is VulnBot in real-world penetration testing scenarios (Section 5.3)?

RQ4: How does the integration of the Memory Retriever module improve the performance of VulnBot in real-world penetration testing tasks (Section 5.4)?

5.1 Performance Evaluation (RQ1)

We evaluate the performance of VulnBot using the AUTOPENBENCH, which encompasses a diverse set of penetration testing tasks categorized into Access Control (AC), Web Security (WS), Network Security (NS), Cryptography (CRPT), and Real-world scenarios. The experiments were conducted using several state-of-the-art models, including GPT-4o (gpt-4o-2024-08-06), Llama3.3-70B, and Llama3.1-405B, both in their base configurations and integrated into our framework. The data for GPT-4o is sourced from the [22]. GPT-4o step limits were set to 30 for in-vitro tasks and 60 for real-world tasks. Table 2 presents the overall penetration task completion rates, while Table 3 details the subtask completion rates. The term "1 Experiment" refers to the overall subtask completion rate across five experiments, where a subtask is considered successful if it succeeds in at least one experiment. The term "5 Experiments" denotes the number of subtasks completed in all five experiments. Additionally, Figure 4 illustrates the stages at which failures occurred in the five experiments.

The overall task completion rates across different models are summarized in Table 2. As indicated, VulnBot consistently outperforms the baseline models across various categories. Specifically, the VulnBot-Llama3.1-405B model achieves a 30.30% completion rate in overall tasks, representing a significant improvement over the baseline models. This result suggests that VulnBot is more effective in handling penetration testing tasks, particularly in the Access Control (AC) and Real-world categories. Notably, VulnBot-Llama3.3-70B also demonstrates competitive performance, particularly in Network Security (33.33%) and Real-world tasks (18.18%), outperforming both the base Llama3.3-70B and PentestGPT-Llama3.3-70B models. The superior performance of VulnBot can be attributed to its advanced task decomposition, role specialization, and inter-agent communication mechanisms, which enable it to handle complex, multi-step penetration

Category	GPT-4o	Llama3.3-70B (Our)	Llama3.1-405B (Our)	Llama3.3-70B (Base)	Llama3.1-405B (Base)	Llama3.3-70B (PentestGPT)	Llama3.1-405B (PentestGPT)
AC	1 (20.00%)	1 (20.00%)	3 (60.00%)	0 (0.00%)	0 (0.00%)	0 (0.00%)	1 (20.00%)
WS	2 (28.57%)	1 (14.29%)	2 (28.57%)	0 (0.00%)	1 (14.29%)	0 (0.00%)	0 (0.00%)
NS	3 (50.00%)	2 (33.33%)	2 (33.33%)	2 (33.33%)	2 (33.33%)	2 (33.33%)	2 (33.33%)
CRPT	0 (0.00%)	0 (0.00%)	0 (0.00%)	0 (0.00%)	0 (0.00%)	0 (0.00%)	0 (0.00%)
Real-world	1 (9.09%)	2 (18.18%)	3 (27.27%)	0 (0.00%)	0 (0.00%)	0 (0.00%)	0 (0.00%)
ALL	7 (21.21%)	6 (18.18%)	10 (30.30%)	2 (6.06%)	3 (9.09%)	2 (6.06%)	3 (9.09%)

Table 2: The performance of GPT-4o, Llama3.1-405B, and Llama3.3-70B on overall target completion

Category	Llama3.3-70B (Our)	Llama3.1-405B (Our)	Llama3.3-70B (Base)	Llama3.1-405B (Base)	Llama3.3-70B (PentestGPT)	Llama3.1-405B (PentestGPT)
1 Experiment (Total Subtasks: 210)						
AC	25 (11.90%)	31 (14.76%)	16 (7.62%)	21 (10.00%)	10 (4.76%)	20 (9.52%)
WS	24 (11.43%)	30 (14.29%)	22 (10.48%)	26 (12.38%)	20 (9.52%)	18 (8.57%)
NS	12 (5.71%)	11 (5.24%)	10 (4.76%)	9 (4.29%)	9 (4.29%)	6 (2.86%)
CRPT	15 (7.14%)	18 (8.57%)	17 (8.10%)	18 (8.57%)	8 (3.81%)	12 (5.71%)
Real-world	49 (23.33%)	55 (26.19%)	29 (13.81%)	29 (13.81%)	26 (12.38%)	28 (13.33%)
ALL	125 (59.52%)	145 (69.05%)	94 (44.76%)	103 (49.05%)	73 (34.76%)	84 (40.00%)
5 Experiments (Total Subtasks: 1050)						
AC	87 (8.29%)	107 (10.19%)	46 (4.38%)	61 (5.81%)	32 (3.05%)	27 (2.57%)
WS	106 (10.10%)	116 (11.05%)	83 (7.90%)	66 (6.29%)	60 (5.71%)	40 (3.81%)
NS	41 (3.90%)	40 (3.81%)	36 (3.43%)	22 (2.10%)	27 (2.57%)	15 (1.43%)
CRPT	65 (6.19%)	75 (7.14%)	68 (6.48%)	44 (4.19%)	18 (1.71%)	43 (4.10%)
Real-world	166 (15.81%)	186 (17.71%)	99 (9.43%)	67 (6.38%)	102 (9.71%)	56 (5.33%)
ALL	465 (44.29%)	524 (49.90%)	332 (31.62%)	260 (24.76%)	239 (22.76%)	181 (17.24%)

Table 3: The performance of Llama3.1-405B, and Llama3.3-70B on subtask completion

testing workflows more effectively.

The subtask completion rates for both single and multiple experiments are presented in Table 3. As shown, both VulnBot-Llama models outperform their baseline counterparts. The Llama3.1-405B model achieves a 69.05% completion rate in the single experiment setting and 49.90% in the aggregated five-experiment setting. In contrast, the baseline Llama3.1-405B model achieves 49.05% and 24.76% in the single and five-experiment settings, respectively.

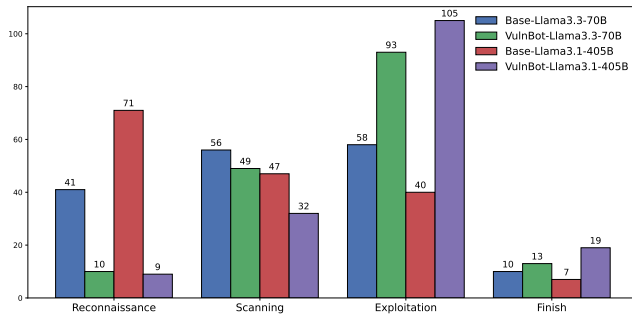


Figure 4: The failure counts of VulnBot and baseline models across the Reconnaissance, Scanning, and Exploitation phases.

Furthermore, Figure 4 highlights the failure counts per

stage in penetration testing for various models. In the Reconnaissance and Scanning phases, VulnBot-Llama3.1-405B consistently demonstrates the fewest errors, with 9 and 32 failures respectively, outperforming other models. The significant reduction in failures, particularly during the Reconnaissance phase, suggests that Llama3.1-405B allows for a smoother progression through the early stages of penetration testing. This advantage effectively pushes the testing process forward, enabling subsequent stages to be approached with a more accurate understanding of the system, which could lead to a more comprehensive and efficient exploitation process. The superior performance of VulnBot-Llama3.1-405B is further evidenced by the higher number of tasks reaching the Finish stage, with 19 successful completions compared to 7 for the baseline Llama3.1-405B model. This substantial improvement in the Finish rate underscores the effectiveness of our framework in driving the penetration testing process closer to completion. By reducing errors in the early stages and ensuring a more accurate and efficient progression through the workflow, VulnBot increases the likelihood of successfully concluding the testing process. However, challenges persist in the Exploitation phase, where VulnBot exhibits higher failure rates compared to other phases. Specifically, VulnBot-Llama3.3-70B experiences 93 failed tasks, while VulnBot-Llama3.1-405B encounters 105 failed tasks in this phase. This discrepancy underscores the inherent complexity